



Iniciándote en Azure DevOps: un proyecto para principiantes

Alba Catoira
Leo Micheloni

En esta charla vamos a hablar de Azure DevOps, y está orientada a todos aquellos que no habéis tenido ocasión de utilizar esta herramienta antes o no sabéis por dónde empezar.

Sponsors

<NetCoreConf/>
Barcelona '25



axazure
by avvale

devoteam

ilitia

Inkoova

NTT DATA

plain
concepts

prodware

intelequia

LDL SCRM
INTERNATIONAL HUB

TOKIOTA

V-Valley
enhancing your business

Venue Sponsors

In collaboration with

Centre d'Estudis
Politécnics

sessionize

Microsoft

Antes de nada, nos gustaría agradecer a los patrocinadores el apoyo a este tipo de charlas e iniciativas de las comunidades técnicas.

¡Hola!

«NetCoreConf»
Barcelona '25



Soy Desarrolladora y Consultora Cloud en Tokiota desde hace 4 años, aunque tengo más de 15 años de experiencia en el desarrollo de software, más concretamente, en tecnologías .Net.

Apasionada del aprendizaje continuo, interesada en todo lo relacionado con la Inteligencia Artificial y novata en eventos tecnológicos.

<https://www.linkedin.com/in/albacatoira/>
alba.catoira@tokiota.com



¡Hola!

«NetCoreConf/»
Barcelona '25



Soy desarrollador de software desde hace más de 20 años, y he sido reconocido como Microsoft MVP los últimos 10 años gracias a mi colaboración con la comunidad de desarrollo de latinoamérica y España. Involucrado con las buenas prácticas de desarrollo y agilidad, y todo lo relacionado con crear software de calidad. Actualmente trabajo para Tokiota en Madrid.

<https://www.linkedin.com/in/leomicheloni>



¿Qué vamos a ver?

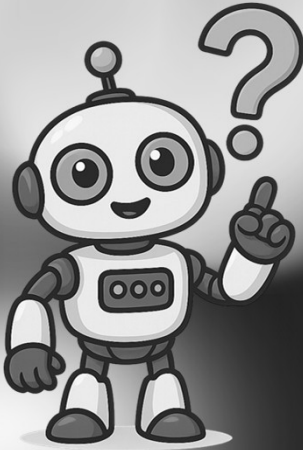
<NetCoreConf/>
Barcelona '25



- 1 Azure DevOps
- 2 Boards
- 3 Repos
- 4 Pipelines
- 5 Tests y Artifacts



Lo que vamos a ver en esta charla es qué es Azure DevOps, y cuáles son los elementos más típicos o más usados de esta herramienta. Siguiendo los pasos de configuración desde cero, con un proyecto de ejemplo muy sencillito



- Práctica de desarrollo de software que promueve la colaboración entre los equipos de desarrollo y de operaciones
 - Entrega de software más rápida y confiable.
 - Conecta las personas, los procesos y la tecnología
- Construido sobre la plataforma de computación en la nube de Azure
- Servicio integral ofrecido por Microsoft
- Existe versión on-premise

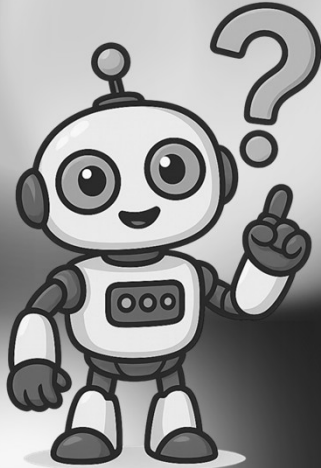
La primera pregunta que debemos hacernos es: ¿qué es Azure DevOps?

DevOps es una práctica de **desarrollo de software** que promueve la colaboración entre equipos y facilita la gestión de proyectos de desarrollo de software, con el fin de lograr una entrega más rápida y confiable.

Construido sobre la plataforma de computación en la nube de Azure, y es un servicio integral ofrecido por Microsoft.

Y Por último, comentar que existe una **versión on-premise**

Esa sería un poco la definición



- Los equipos de desarrollo y de operaciones no están sincronizados,
 - Entrega del software más lenta.
- DevOps añade eficiencia en todas las tareas implicadas en el desarrollo, la implementación y el mantenimiento de software.
 - Aumenta la visibilidad
 - Requisitos más exactos
 - Mejora la comunicación
 - Reduce el tiempo de comercialización.

¿Por qué es necesario Azure DevOps?

No es que sea algo obligatorio, pero su uso está muy extendido hoy en día, en muchos clientes y en muchos proyectos, por lo que profesionalmente es muy recomendable tener conocimientos de esta práctica.

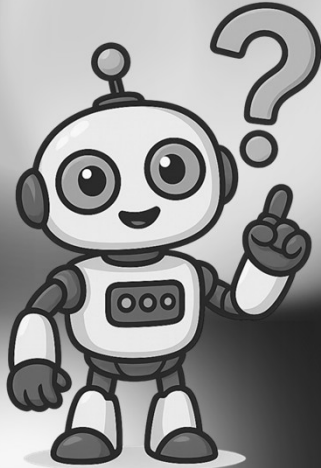
“Cuentan que hace unos años (esto ya no pasa, por supuesto) te pasaban las tareas en una reunión, o tenías que desplegar el código comprimido en un zip, o sobrescribir algún exe o alguna dll”. Pues este tipo de cosas son las que hemos dejado atrás gracias a DevOps.

El proceso de desarrollo de software puede implicar una gran cantidad de trabajo manual y dar lugar a un número importante de errores en el código. A menudo, los equipos de desarrollo y de operaciones no están sincronizados, lo que ralentiza la entrega del software y decepciona al cliente.

DevOps pretende añadir eficiencia en todas las tareas implicadas en el desarrollo, la implementación y el mantenimiento de software, conectando a los equipos de desarrollo y de operaciones.

¿En qué se diferencia DevOps de otras prácticas de desarrollo de software?

<NetCoreConf/>
Barcelona '25



- DevOps salva el vacío que existe entre los equipos de desarrollo y de operaciones
 - Aumenta la eficiencia en el proceso de desarrollo e implementación de software.
- DevOps hace un fuerte hincapié en la **automatización**, lo que reduce el número total de errores.

La siguiente pregunta es: ¿En qué se diferencia DevOps de otras prácticas de desarrollo software?

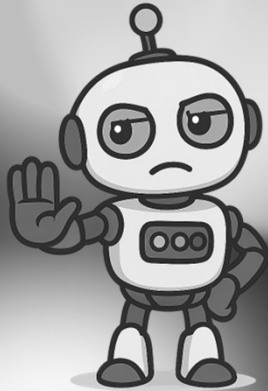
Como decíamos antes, DevOps pretende salvar el vacío de comunicación que existe entre los equipos de desarrollo y de operaciones, y hace un especial hincapié en la **automatización de las fases del proceso de desarrollo**, lo que reduce el número total de errores.

Es una práctica que está muy ligada a la metodología Ágil, de forma que se puedan estar integrando cambios todo el tiempo. Esto hace referencia a la integración continua: tenemos todo el tiempo la última versión desplegada y funcionando y la entrega se hace mucho más rápida.

Y por último, la última pregunta es:

¿Azure DevOps y Azure son lo mismo?

<NetCoreConf/>
Barcelona '25



- Aunque se llama Azure DevOps, no tiene que ver directamente con Azure.
- Multiplataforma

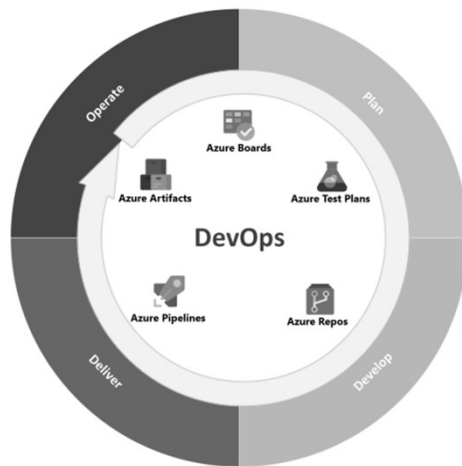


¿Azure DevOps y Azure son lo mismo? Pues no

Aunque se llama Azure DevOps, no tiene que ver directamente con Azure. Es decir, obviamente, puedes desplegar tu proyecto en Azure, y ese sería quizás el escenario más común, pero también podrías hacerlo en AWS, Google Cloud, utilizar Jenkins, etc

Es decir, que Azure DevOps es multiplataforma.

Y después de haber resuelto todas estas preguntas básicas, vamos a meternos de lleno en qué es lo que Azure DevOps nos ofrece



Azure Boards



Azure Repos



Azure Pipelines



Test Plans



Artifacts

Nosotros vamos a centrarnos en estos 5 servicios, que son de los más importantes y vamos a ir viendo cómo y cuándo utilizarlos a lo largo de nuestro proyecto:

- Azure Boards
- Azure Repos
- Azure Pipelines
- Test Plans
- Azure Artifacts



Azure Boards



- Nuestro proyecto: API .NET8
- Requerimientos: Crear un API que consulte una base de datos y permita realizar las operaciones CRUD esenciales sobre alguna tabla



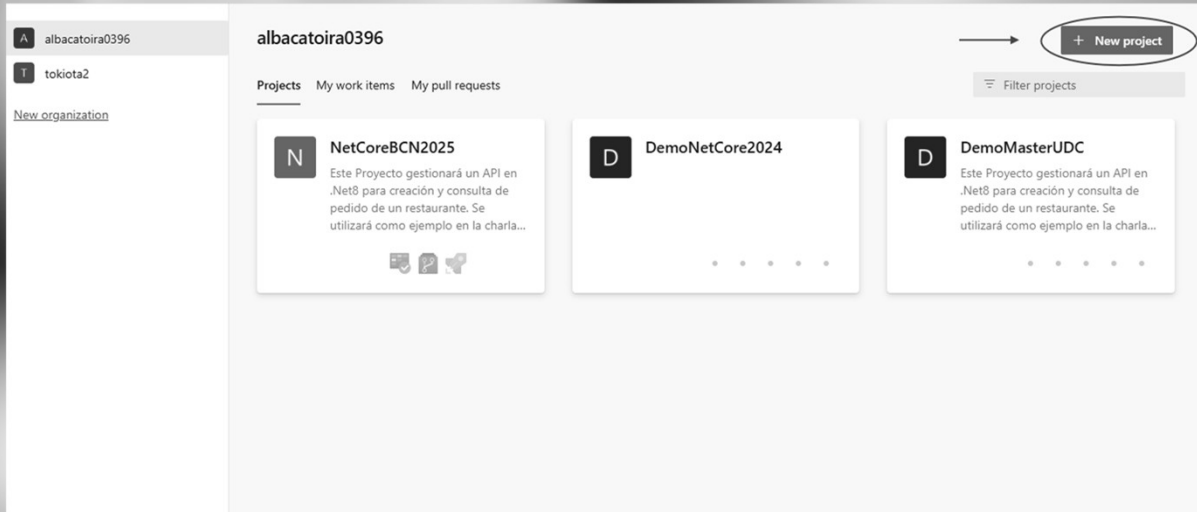
<https://github.com/acatoirafranco/DemoNetCore2024>

Azure Boards es la herramienta que nos permite llevar un seguimiento de tu backlog y controlar el progreso de tu proyecto

En nuestro caso vamos a crear una API mínima en Net8, por ahora con dos métodos, un GET y un POST para obtener y crear Pedidos de un Restaurante

Azure Boards: Creación de un proyecto

<NetCoreConf/>
Barcelona '25



¿Cómo empezamos a trabajar con Azure Boards? Lo primero que tenemos que hacer cuando entramos en Azure DevOps en nuestra organización es **crear nuestro proyecto**

Esta es la interfaz con la que nos encontramos al entrar en una organización en Azure DevOps, así que le daremos a **Crear Nuevo Proyecto** e introduciremos un nombre para nuestro proyecto.

Azure Boards: Creación de un proyecto

- Control de código fuente
 - No tiene por qué ser DevOps. Puede ser repositorio externo
- Work Item Process: Metodología de trabajo

«NetCoreConf/»
Barcelona '25

Create new project

Project name *

NetCoreConfBCN2025

Description

Este Proyecto gestionará un API en .Net8 para creación y consulta de pedido de un restaurante.
Se utilizará como ejemplo en la charla del Máster de la NetCoreConf2025

Visibility

Public
Anyone on the internet can view the project. Certain features like TFVC are not supported.

Private
Only people you give access to will be able to view this project.

Advanced

Version control ¹²

Git

Work item process ¹²

Agile

Cancel Create

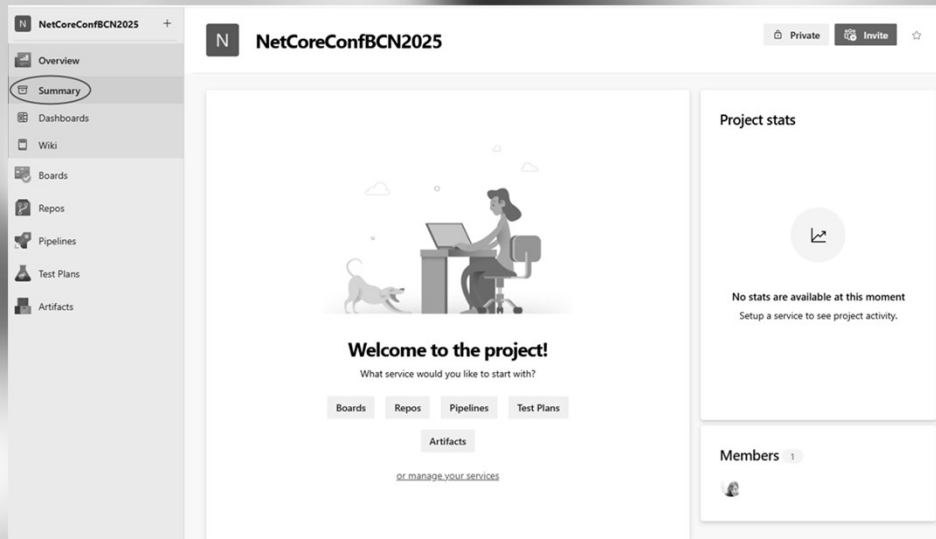
También seleccionaremos la visibilidad que queremos darle, en este caso Privada, y en las opciones avanzadas, elegiremos el tipo de control de código fuente, que en la mayoría de los casos, será Git.

Aquí es importante destacar que el repositorio no tiene que ser de DevOps, si no que puede ser algún repositorio externo.

También podremos seleccionar la metodología de trabajo de nuestros procesos. En nuestro caso, vamos a seguir una metodología Agile.

Azure Boards: Creación de un proyecto

<NetCoreConf/>
Barcelona '25

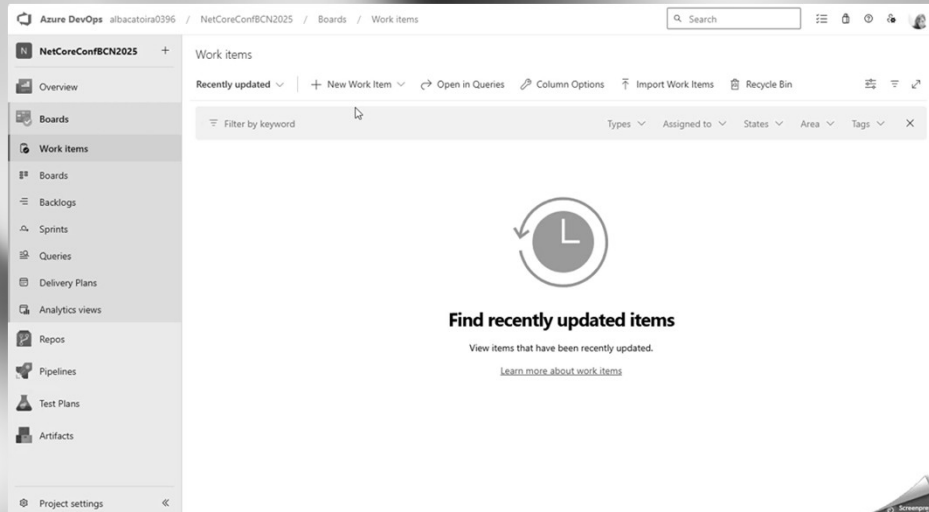


Y ya tenemos nuestro panel de Inicio en Azure DevOps para nuestro proyecto

Esta pantalla principal se puede además customizar con el estado del proyecto: builds, estadísticas de test, etc

Azure Boards: Crear elementos

<NetCoreConf/>
Barcelona '25



Una vez creado el proyecto, nos vamos a ir directos a la sección **Boards**, y vamos a ir creando elementos en nuestro backlog. Podemos comenzar creando un Nuevo WorkItem de tipo User Story:

Y en nuestra User Story, pondremos el título, por ejemplo, “Creación de Proyecto API”, y en la descripción, escribiremos lo que dijimos antes: como usuario, quiero tener un api que acceda a la base de datos previamente creada y que permita realizar las operaciones CRUD esenciales sobre una tabla

Cuanto más acotemos la descripción mas fácil será comprobar los criterios de aceptación. Que en este caso serán:

- El Api debe estar bajo un control de versiones
- El API debe contar con dos métodos iniciales
- El API debe de tener un mínimo de pruebas automatizadas

Aquí cabe decir que en general, suele ser el Product Manager quien se encargue de rellenar tanto la descripción como los criterios de aceptación de las User stories. Debe ser él quien plasme los requerimientos del cliente en los elementos del backlog de manera eficiente.

Azure Boards: Crear User Story

<NetCoreConf/>
Barcelona '25



Azure DevOps albacatoira0396 / NetCoreBCN2025 / Boards / Work items

NetCoreBCN2025

Recently updated Back to Work Items 2 of 2

USER STORY 14

14 Creación de Infraestructura

No one selected 0 Comments Add Tag Save Follow

Updated by Alba Catoira Franco Yesterday

Details

Description

creación de infraestructura para el API. Como desarrollador, quiero poder desplegar de forma automatizada la última versión del proyecto

Acceptance Criteria

Deben estar creados todos los recursos cloud que se necesitan para el API
Todos los procesos deben estar automatizados mediante pipelines

Discussion

Add a comment. Use # to link a work item, @ to mention a person, or ! to link a pull request.

switch to Markdown editor

Planning

Story Points

Priority 2

Risk

Classification

Value area Business

Deployment

To track releases associated with this work item, go to Releases and turn on deployment status reporting for Boards in your pipeline's Options menu. [Learn more about deployment status reporting](#)

Development

Add link

Link an Azure Repos commit, pull request or branch to see the status of your development. You can also [create a branch](#) to get started.

Related Work

Add link

Volvemos al backlog y creamos una nueva User Story para crear nuestra infraestructura.

Como descripción ponemos que quiero poder desplegar de forma automatizada la última versión de nuestro proyecto, y como criterios de aceptación pondremos que deben estar creados todos los recursos cloud que se necesitan para el API, y que todos los procesos deben estar automatizados mediante pipelines

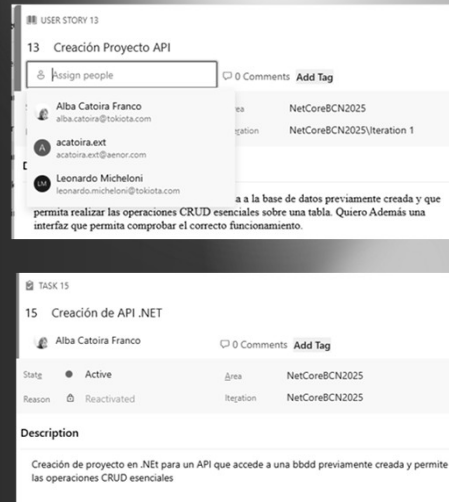
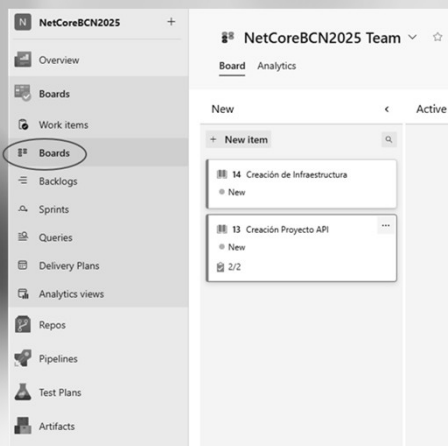
Backlog		Analytics			
		Order	Work Item Type	Title	State
+	1	User Story		 Creación Proyecto API  Creación de API .NET  Creación de tests	● Active ● Closed ● Closed
	2	User Story		 Creación de Infraestructura	● New

Elegimos crear una tarea de tipo Child, le ponemos un título.

- 1) O bien como tareas hijas dentro de la User Story
- 2) O si volvemos al backlog o a la pestaña de Boards, veremos que nos aparecen las US anidadas con sus tareas inferiores

Azure Boards: Asignar elementos

<NetCoreConf/>
Barcelona '25



Por último, ya tenemos creadas nuestras User Stories y nuestras tareas, y lo que faltaría sería asignarnos las tareas que vamos a abordar nosotros. Para ello, volvemos a Boards, y seleccionamos al User Story. En el combo de usuarios, donde ahora pone Desasignado, elegimos al usuario del equipo que va a abordar el trabajo, en este caso, a nosotros.

Con esto ya hemos visto lo más importante para empezar a gestionar el backlog de nuestro proyecto



Azure Repos

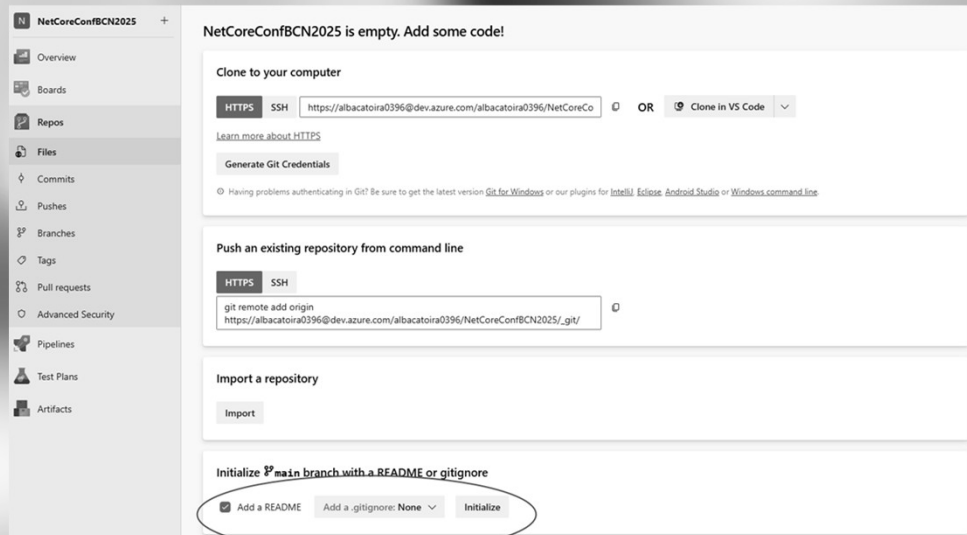


- Herramienta para alojar los archivos de nuestro proyecto
- Control de código fuente

Azure Repos es la herramienta que alojará todos los archivos de nuestro proyecto, que puede ser código o puede ser documentación. Es donde gestionaremos el control del código fuente

Azure Repos: Inicializar repositorio

<NetCoreConf/>
Barcelona '25

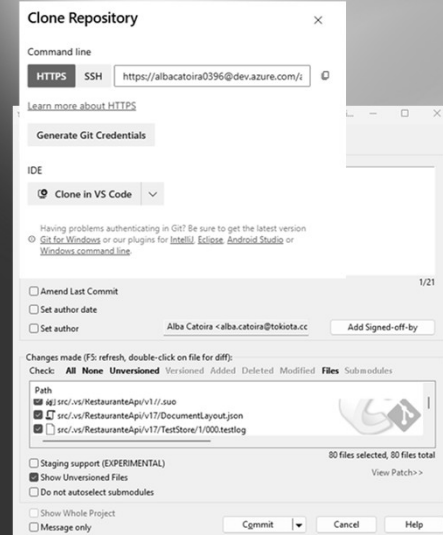
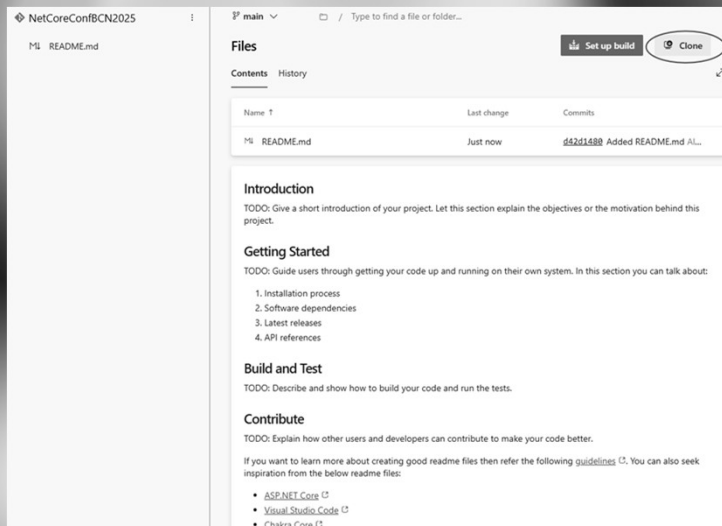


La primera vez que entramos en **Repos**, podemos ver que el repositorio está vacío.

En la parte inferior de la página principal, podemos **inicializar** una rama **main** con un README o gitignore. Le damos a inicializar, y con esto, creamos un repositorio con una rama main por defecto

Azure Repos: Crear repositorio

<NetCoreConf/>
Barcelona '25



Y tendremos un repositorio que coincide con el nombre del proyecto.

- 1) Si le damos a Clonar, podremos copiar la url e irnos a un terminal para ejecutar un git clone, o ejecutar algún cliente tipo TortoiseGit para clonar el repositorio en nuestro equipo. Dentro de ese proyecto, bien desde el explorador o bien desde un Visual Studio. En nuestro caso, hemos usado un cliente de tipo Tortoise para clonar el repositorio en nuestro equipo

Ahora necesitamos tener código que subir. En nuestro caso, hemos creado en Visual Studio un API mínimo para pedidos de un restaurante, y hemos subido los cambios al repositorio (con el comando git push, o con el Visual Studio, o el Tortoise)

Azure Repos: Subir código

<NetCoreConf/>
Barcelona '25



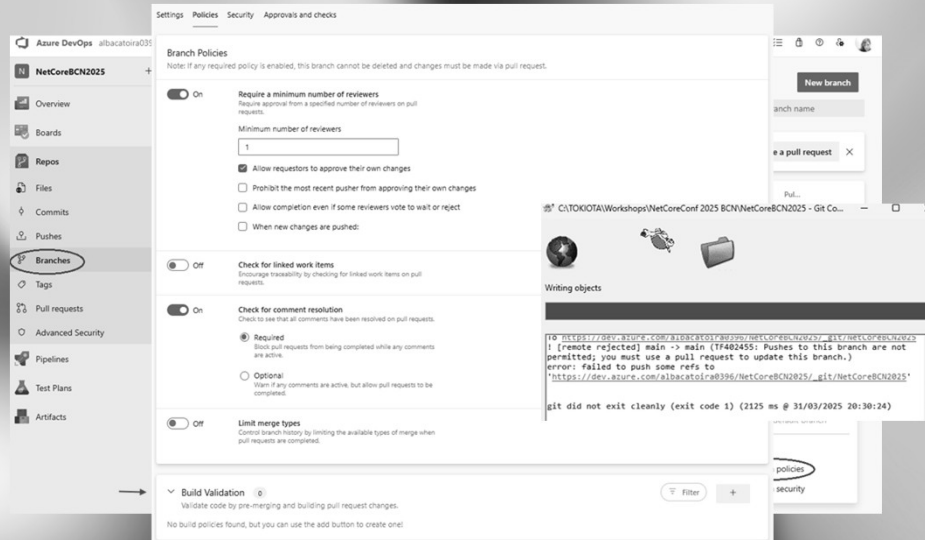
The screenshot shows the Azure Repos web interface for a repository named 'NetCoreBCN2025'. The left sidebar shows the file structure with folders 'infra', 'pipelines', and 'src', and a file 'README.md'. The main area shows the 'Files' section with a table of recent commits.

Name	Last change	Commits
infra	Yesterday	a82a46e8 First version of API Alb...
pipelines	Yesterday	a82a46e8 First version of API Alb...
src	Yesterday	a82a46e8 First version of API Alb...
README.md	Yesterday	85dc489b Added README.md AL...

Below the table, there are sections for 'Introduction' and 'Getting Started' with placeholder text (TODO).

Después de hacer un Push al repositorio con nuestro código, si accedemos de nuevo a la sección **Repos** de Azure DevOps, veremos que ya tenemos el código subido.

Pero si os fijáis, **hemos subido nuestro código sin que haya sido revisado ni validado por nadie, por lo que es importante crear una política de ramas.**



1) Al ir de nuevo a la sección Repos, vemos que sólo tenemos nuestra rama main. Pinchamos en los 3 puntitos de la derecha y elegimos la opción **Branch Policies**.

2) Activamos la opción para que **haya un mínimo número de aprobadores**, que en este caso, vamos a poner sólo 1, pero marcamos la opción de **prohibir que nosotros mismos nos aprobemos los cambios**, y que si se hace un commit de nuevos cambios, estos tengan que ser aprobados de nuevo.

Existen otras opciones que podemos activar, como que los comentarios de los compañeros deban ser resueltos antes de completar la Pull request, o que cada commit deba llevar un link con un item del backlog relacionado. En este caso, lo vamos a dejar así.

3) Si ahora volvemos al código, hacemos cualquier cambio, e intentamos hacer un commit y un push de nuevo a la rama main, nos saldrá un mensaje similar a este, indicando que ya no se pueden hacer cambios directamente a la rama main, sin hacer una Pull request desde otra rama

Azure Repos: Cerrando tareas

<NetCoreConf/>
Barcelona '25



TASK 15*

15 Creación de API .NET

Alba Catoira Franco 0 Comments Add Tag

State: **Closed** Area: NetCoreBCN2025

Reason: Completed Iteration: NetCoreBCN2025

Creación de proyecto en .NET para un API que accede a una bdd previamente creada y permite las operaciones CRUD esenciales

TASK 16

16 Creación de tests

Alba Catoira Franco 0 Comments Add Tag

State: **Closed** Area: NetCoreBCN2025

Reason: Completed Iteration: NetCoreBCN2025

USER STORY 13*

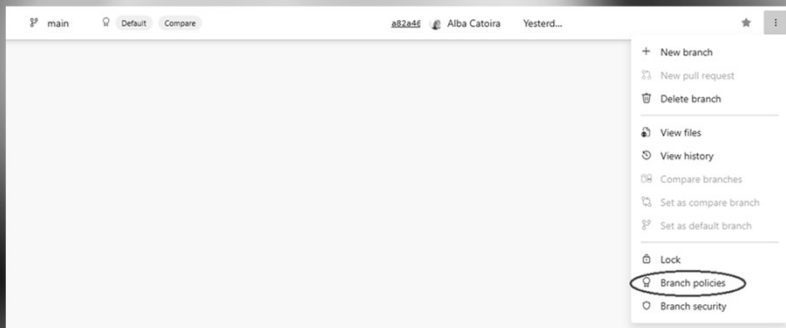
13 Creación Proyecto API

Alba Catoira Franco 0 Comments Add Tag

State: **Closed** Area: NetCoreBCN2025

Reason: Acceptance tests pass Iteration: NetCoreBCN2025/Iteration 1

Como ya hemos subido nuestro código del API, ahora podremos volver a la sección de **Boards**, donde teníamos una tarea específica para la creación de nuestro API, y ya podemos marcar esta tarea como **Cerrada**



Build Validation 0

Validate code by pre-merging and building pull request changes.

No build policies found, but you can use the add button to create one!

Filter



Crearemos nuestros Pipelines antes de configurar esta sección



Azure Pipelines



- Es la herramienta para desplegar nuestro software a los diferentes entornos y poder saber qué desplegamos y cómo lo hacemos.
- Concepto de CI/CD (integración continua y despliegue continuo)
- Organizar valores de configuración por entornos
- Desplegar con un click
- Se pueden programar despliegues o validaciones (nightly builds)

Azure Pipelines es la herramienta que utilizaremos para desplegar nuestro software a los diferentes entornos y poder saber qué desplegamos y cómo lo hacemos.

Sabemos en todo momento qué versiones tenemos desplegadas en cada entorno, y podemos controlar el pasar una versión de un entorno a otro, así como volver atrás a la versión anterior más fácilmente.

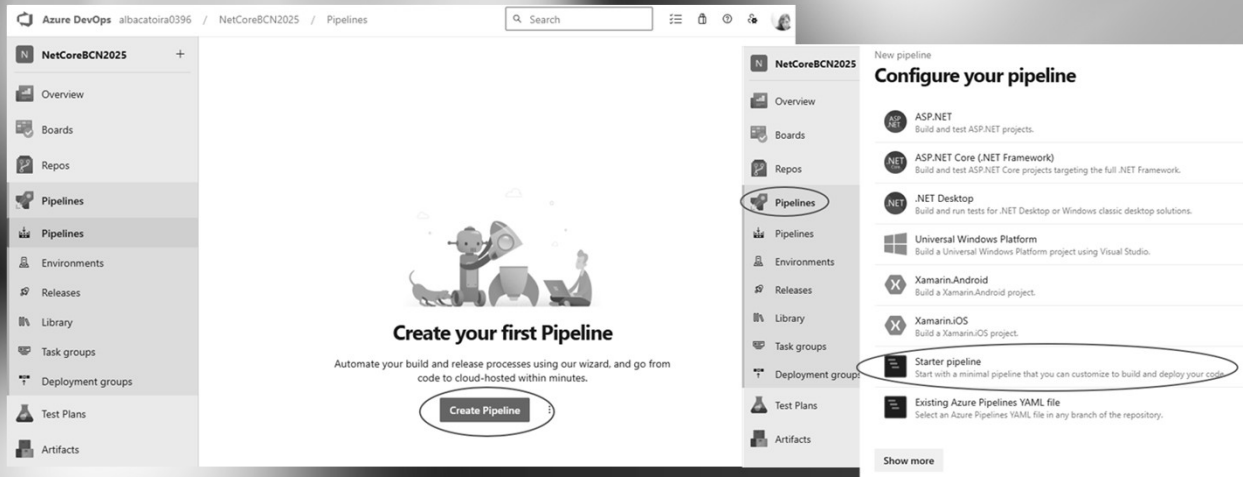
Es aquí donde se lleva a cabo el concepto de CI/CD (integración continua y despliegue continuo)

Esto nos va a permitir cosas como:

- Organizar los valores de configuración de cada entorno
- Desplegar con un click a partir de la aprobación de una PR
- Se pueden programar despliegues o validaciones

Azure Pipelines: Creando un pipeline

<NetCoreConf/>
Barcelona '25



Al acceder a la sección **Azure Pipelines** la primera vez, y darle a **crear uno nuevo**, estas son las opciones que nos aparecen.

1) En nuestro caso, crearemos uno a partir de un **Repositorio Git**, que acabamos de crear, y después seleccionamos la opción **Starter pipeline**

Azure Pipelines: Creando un pipeline

<NetCoreConf/>
Barcelona '25



New pipeline Review your pipeline YAML

NetCore8CN2025 pr-validator.yml

```
1 # Starter pipeline
2 # Start with a minimal pipeline that you can customize to build and deploy your code.
3 # Add steps that build, run tests, deploy, and more:
4 # https://aka.ms/yaml
5
6 trigger:
7 - main
8
9 pool:
10 - vmImage: ubuntu-latest
11
12 steps:
13 - script: echo Hello, world!
14   displayName: 'Run a one-line script'
15
16 - script: |
17   echo Add other tasks to build, test, and deploy your project.
18   echo See https://aka.ms/yaml
19   displayName: 'Run a multi-line script'
20
```

```
trigger: none
pool:
  vmImage: 'ubuntu-latest'
variables:
- task: DotNetCoreCLI@2
  displayName: 'Unit and Integration Test'
  inputs:
    command: test
    projects: '**/*RestaurantApiTests.csproj'
    arguments: '--configuration $(buildConfiguration) --collect "XPlat Code coverage" --logger trx'
  - script: 'reportgenerator -reports:$(Agent.TempDirectory)**/coverage.cobertura.xml -targetdir:$(Build.SourcesDirectory)/coverlet'
  displayName: 'Create reports'
Settings
- task: PublishTestResults@2
  condition: succeededOrFailed()
  displayName: Publish test results
  inputs:
    testRunner: VSTest
    testResultsFiles: '**/**.trx'
    searchFolder: '$(Agent.TempDirectory)'
Settings
- task: PublishCodeCoverageResults@2
  condition: succeededOrFailed()
  displayName: Publish code coverage
  inputs:
    codeCoverageTool: Cobertura
    summaryFileLocation: '$(Build.SourcesDirectory)/coverlet/reports/Cobertura.xml'
```

Nos aparecerá un código yaml básico, por defecto, que tendremos que ir adaptando a nuestro proyecto

- 1) Podemos cambiar el nombre al archivo yml, y en nuestro caso, como **crearemos un validador que se ejecutará incluso antes de hacer la pr a nuestra rama y el merge del código**, lo llamaremos **pr-validator**.

LEO: ATENTO SI ME OLVIDO

Como parte de una validación de una Pull Request, se puede poner un “build” que verifique cosas. También se pueden automatizar controles de calidad:

- **test,**
- **cobertura,**
- **buenas prácticas,**
- **seguridad**

Una vez se hayan ejecutado y pasado todas estas tareas de este pipeline, podremos decir que el código puede pasar a integrarse en la rama main

- **CONSEJO:** Esta parte quizás es la que más intimida, sobre todo si no estáis acostumbrados a escribir código yaml, pero no tengáis miedo de probar y de

metros porque no es tan difícil como parece la primera vez

2a) Una vez dentro del código yaml, lo que haremos será

- Indicar que no habrá ningún desencadenante que ejecute el pipeline (**trigger none**)
- Vamos a utilizar una **imagen con la última versión de Ubuntu**
- Y crearemos una variable de tipo Release

2b) Y a continuación **podemos crear todo el proceso por medio de stages o etapas.**

- Crearemos una etapa que llamamos de **Validación**, con un trabajo que constará de **una tarea de build apuntando al directorio src donde tenemos el código**
- Luego crearemos una tarea para **instalar el generador de informes de test** para después

3) Luego crearemos una tarea de **test**, que ejecute y valide que los test que hemos implementado son correctos

* Otra que me permita **almacenar los resultados en un archivo** de forma que luego pueda publicarlo al finalizar el pipeline

- Lo siguiente que sigue es **crear una tarea para la publicación de los resultados y estimar la cobertura del código**
- Ya sólo nos faltan las tareas para desplegar nuestro API....



```

Settings
- task: DotNetCoreCLI@2
  displayName: dotnet-restore
  inputs:
    command: restore
    projects: '**/*RestauranteApi.csproj'

Settings
- task: DotNetCoreCLI@2
  inputs:
    command: publish
    projects: '**/*RestauranteApi.csproj'
    publishWebProjects: false
    arguments: '--configuration $(BuildConfiguration) --output $(Build.ArtifactStagingDirectory)'
    zipAfterPublish: false
    modifyOutputPath: false

Settings
- task: ArchiveFiles@2
  displayName: 'Zip publish folder'
  inputs:
    rootFolderOrFile: '$(Build.ArtifactStagingDirectory)'
    includeRootFolder: false
    archiveType: 'zip'
    archiveFile: '$(Build.ArtifactStagingDirectory)/$(Build.BuildNumber).zip'
    replaceExistingArchive: true

Settings
- task: PublishBuildArtifacts@1
  displayName: 'Publish Build Artifact'
  inputs:
    PathToPublish: '$(Build.ArtifactStagingDirectory)'
    ArtifactName: 'drop'
    publishLocation: 'Container'

Settings
- task: AzureWebApp@1
  displayName: 'Azure Web App'
  inputs:
    azureSubscription: 'acatoira-subscription-service-connection'
    appType: 'webAppLinux'
    appName: 'acatoira-restaurant'
    resourceGroupName: 'ACATOIRA'
    package: '$(Build.ArtifactStagingDirectory)**/*.zip'
    
```

1) Lo siguiente será

- crear una tarea de tipo publish para publicar los resultados generados por el proyecto
- Y comprimir los archivos generados en un zip con la intención de copiarlos luego en el deploy

CONSEJO: Esta es quizás la parte más complicada en cuanto a la configuración, ya que cada proyecto tendrá unas necesidades diferentes, y en función del tipo de despliegue, habrá que buscar la task adecuada para el yml. Pero podeis usar el asistente en el editor de yamls, y una vez sepáis el nombre de la task, buscar mas información en tutoriales y foros.

Bueno, y por último,

- La siguiente tarea publica ese artefacto de compilación en Azure Pipelines
- Y una tarea AzureWebApp que **realiza el deploy propiamente dicho**, y publica los archivos del zip en un App Service, donde tendremos que indicarle la **service connection** de nuestro proyecto (y que aún no tenemos y que crearemos después), el **tipo de aplicación** que será, en nuestro caso una webAppLinux, el **nombre de la app** (acatoira-restaurant, que también tendremos que crear) y el **grupo de recursos donde estará alojado** (ACATOIRA)

Azure Pipelines: Creando un pipeline

<NetCoreConf/>
Barcelona '25



The screenshot displays the Azure Pipelines web interface. On the left, a YAML configuration file named 'pr-validatory.yml' is shown with line numbers 65 to 93. The configuration includes tasks for archiving files, publishing artifacts, and deploying to Azure App Service. On the right, a 'Tasks' panel lists various predefined tasks such as 'Android signing', 'Ant', 'App Center distribute', 'App Center test', 'Archive files', 'ARM template deployment', 'Azure App Configuration Export', 'Azure App Configuration Import', 'Azure App Configuration Snapshot', 'Azure App Service deploy', and 'Azure App Service manage'.

- Sé que esto de ver el yaml y ver los nombres de las tareas, puede asustar, pero lo que podéis hacer, sobre todo al principio, **es usar el asistente de tareas**, y como podemos ver, existen muchos tipos de tareas predeterminadas. Pueden ser de tipo **script**, de tipo **bash**, agregar un tag al código, y muchos tipos más. Entre el asistente, que ya os da una descripción de para qué sirve cada una, y la documentación que hay disponible en la red, no deberíais tener problema.
- La recomendación es buscar si existe una tarea predeterminada para lo que quieres hacer, y si no, pues puedes crear un script con lo que quieras ejecutar



Save ×

Saving will commit azure-pipelines.yml to the repository.

Commit message

Set up CI with Azure Pipelines

Optional extended description

Add an optional description...

☐ Commit directly to the main branch

☒ Create a new branch for this commit

azure-pipelines

Based on the main branch

☒ Start a pull request

TASK 19

19 Crear Pipeline para desplegar container app

Alba Catoira Franco 0 Comments [Add Tag](#)

State ● Active **Area** NetCoreBCN2025

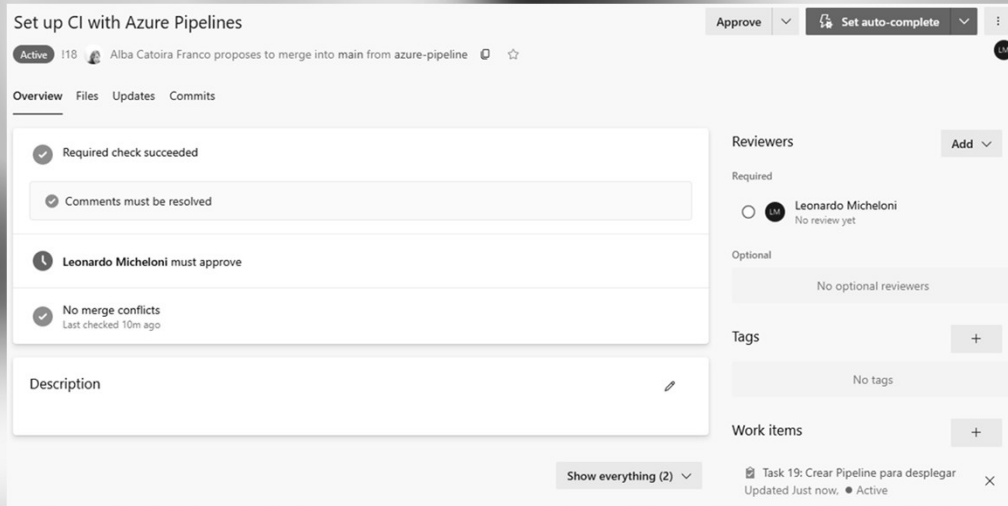
Reason 🏠 Work started **Iteration** NetCoreBCN2025\Iteration 1

Description

Crear código para el pipeline que despliega la infra

Cuando vamos a guardar el pipeline, si intentásemos hacer un commit directamente a main, nos daría un error por la política de ramas que hemos activado, así que **seleccionamos la opción de crear una nueva rama con el cambio**.

2) Si recordamos, teníamos una task en el **Board** para crear una pipeline así que puede ser una buena práctica nombrar la rama con el nombre del autor, o con el nombre del entorno del que parte, y el número de item del board al que hace referencia (en este caso, el 5).



Al guardar los cambios en el pipeline, se crea automáticamente un Pull Request que podemos ver en la sección de Repos de Azure Devops, donde tenemos como revisor (en este caso opcional) a Leo, y donde hemos añadido el item relacionado con esta tarea. Cuando Leo aprueba la PR, los cambios se habrán fusionado en la rama main.

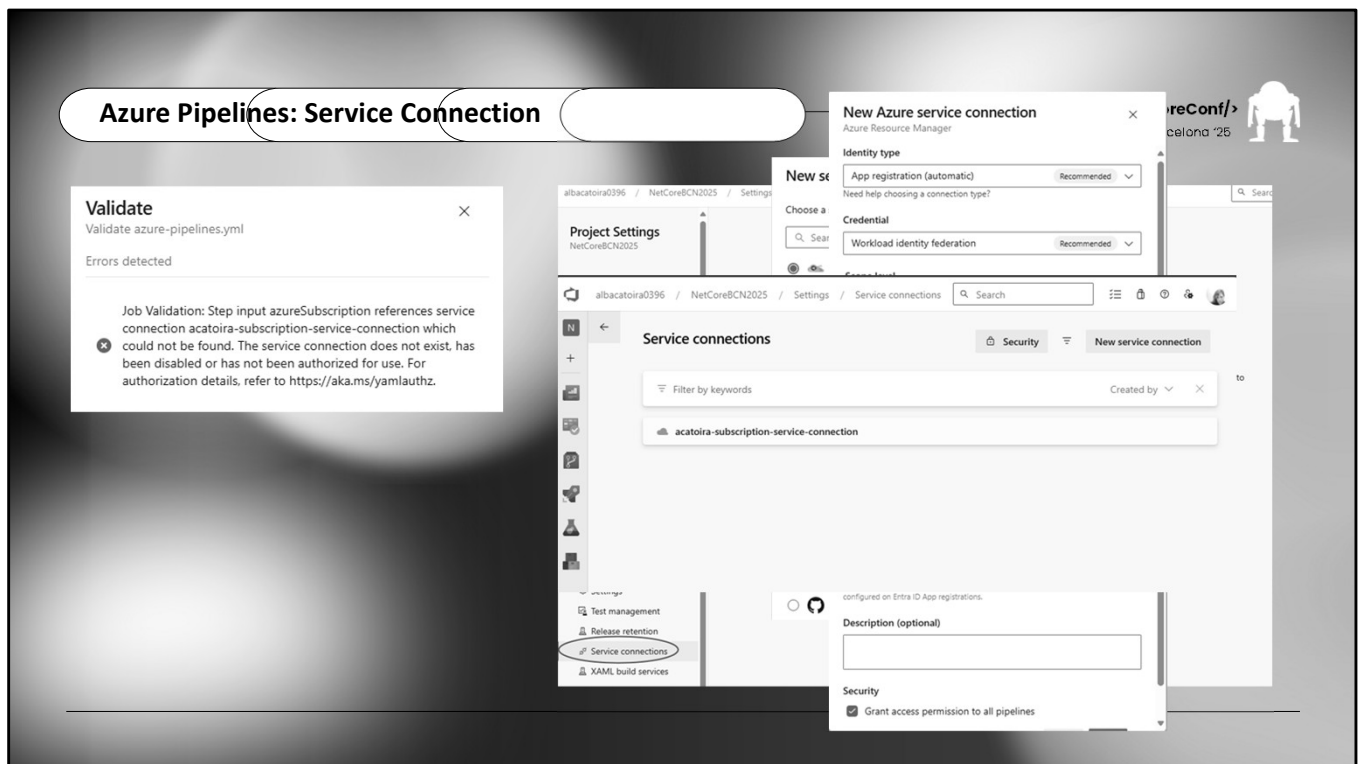
En este momento, como no tenemos el service connection, ni los datos de los recursos del portal de Azure donde vamos a desplegar, no funcionará el pipeline.



- Ejecutar cada vez que se actualice la rama
- El Build debe ser correcto para que se complete la Pull Request

Si recordáis, antes en la sección de Repos habíamos dejado sin configurar una sección que se llamaba Build Validation, porque no teníamos nuestro pipeline. Ahora ya tenemos una versión inicial, así que podemos volver a esa sección.

Si volvemos a la rama **main** en Repos, y seleccionamos **las Branch Políticas**, en **Build Validation** podremos seleccionar el pipeline que acabamos de crear, e indicar que se ejecute al actualizar la rama, y que el Build debe ser correcto para que se complete la pull request



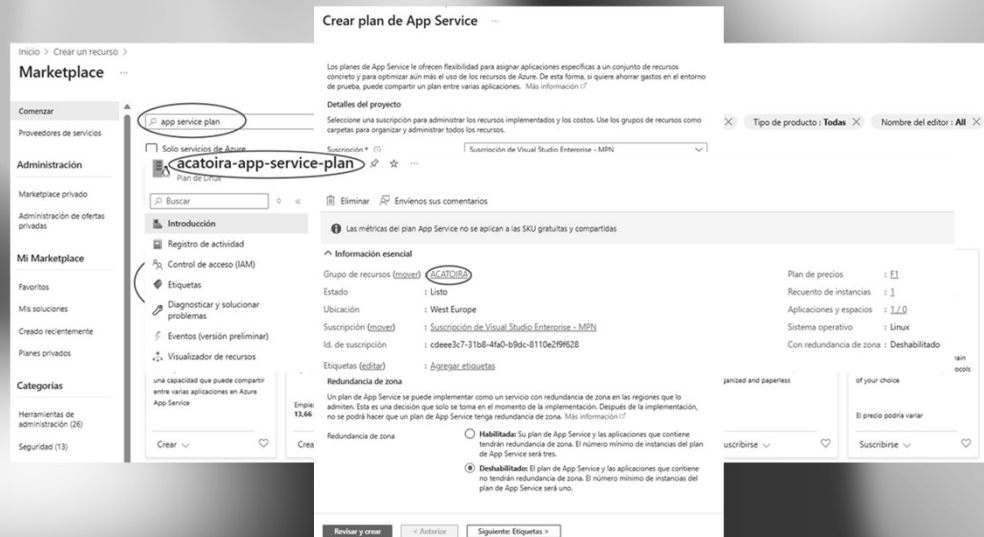
¿Qué nos falta para desplegar nuestro api y que el pipeline termine correctamente?
Pues nos faltan los recursos donde se va a alojar todo.

Para desplegar nuestro api, podríamos usar Terraform y crear un pipeline para el despliegue con Terraform, o en **nuestro caso, vamos a crear un App Service vinculado a un App Service Plan**, y adaptar nuestro yml para que despliegue el api automáticamente con los valores que necesitemos.

En nuestro caso, **lo primero que haremos será crear un Service Connection en nuestro Azure Devops**

- 1) desde Azure Devops, vamos a la **sección Project Settings, y creamos un Service connection seleccionando la opción de Azure Resource Manager**. El nombre de este service connection es el que nos hará falta después en la configuración de despliegue y en nuestro yaml.

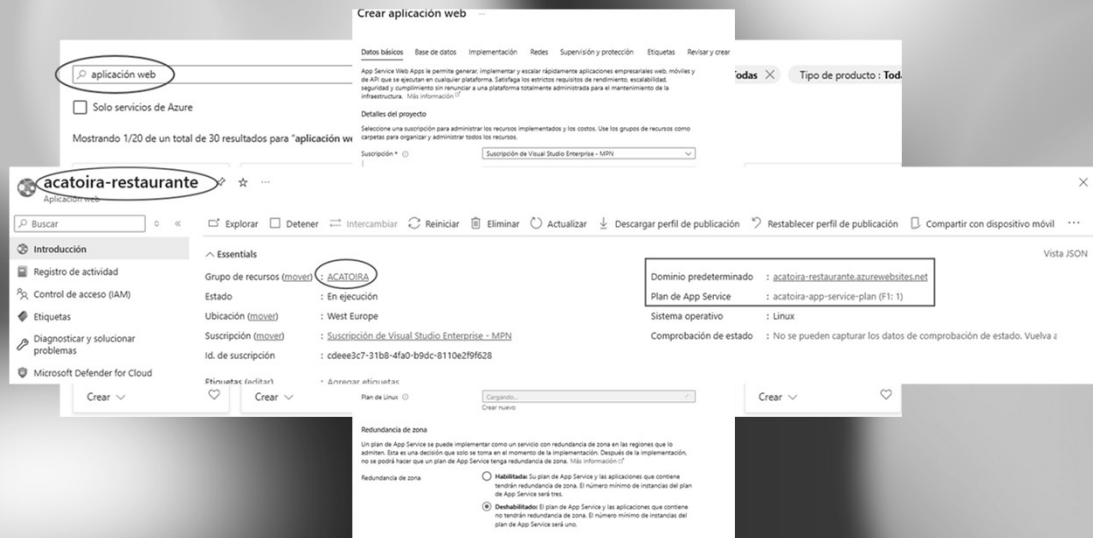
Una Service Connection de tipo Azure Resource Manager (ARM) en Azure es una configuración que permite a tus aplicaciones o servicios conectarse y autenticar de manera segura con los recursos de Azure. Esencialmente, actúa como un puente que facilita la comunicación entre tu entorno de desarrollo o de implementación y los recursos que tienes en Azure, como máquinas virtuales, bases de datos, y más.



Luego, tendremos que acceder al portal de Azure (no de Azure DevOps) para crear un **App Service Plan**, vinculado a una suscripción y en un grupo de recursos,

Entonces, dentro de nuestro grupo de recursos de Azure, si le damos a **Crear**, se nos abre un buscador de elementos Azure. Ponemos app service plan, y elegimos el recurso correspondiente.

Dentro del app service plan tendremos que elegir uno de los planes que nos ofrece, según las necesidades que vaya a tener nuestro proyecto, y los costes asociados, claro. En nuestro caso, hemos elegido un plan gratuito que tendrá ciertas limitaciones.



Una vez creado el service plan, crearemos el **App Service** para alojar nuestro **API**, también bajo la misma suscripción y en el mismo grupo de recursos, y vinculado al plan que acabamos de crear.

Dentro de las opciones, se nos permite configurar la **plataforma de destino de nuestra aplicación (en nuestro caso Net8)**, si queremos usar http o https, y algunos parámetros más. Una vez creado el app service lo que necesitaremos para nuestro yaml es el nombre, y tendremos también la url en la que estará desplegado el API.

Con estos datos, podemos modificar nuestra versión temporal del yaml del pipeline, y guardarlo de nuevo. Subimos los cambios, y hacemos una nueva pull request.

Azure Pipelines: Ejecución del pipeline

<NetCoreConf/>
Barcelona '25



```
Settings
- task: AzureWebApp@1
  displayName: 'Azure Web App'
```

Pipeline is valid

Run new

#20250402.1 • Set up CI with Azure Pipelines

NetCoreBCN2025

This run is being retained as one of 3 recent runs by pipeline.

View retention leases

Summary Tests Code Coverage

Pull request by Alba Catoira Franco

View change

Repository and version	Time started and elapsed	Related	Tests and coverage
NetCoreBCN2025	Today at 18:28	0 work items	100% passed
18 3ef885ee	2m 13s	2 published	65.89% covered

Jobs

Name	Status	Duration
Validation	Success	2m 2s

Jobs in run #20250402.1

NetCoreBCN2025

Net_Validate

Validation	2m 2s
Initialize job	13s
Checkout NetCoreBCN...	1s
Use .NET Core sdk	9s
Install ReportGenerator	2s
Unit and Integration T...	29s
Create reports	< 1s
Publish test results	4s
Publish code coverage	3s
dotnet restore	3s
DotNetCoreCLI	4s
Zip publish folder	2s
Publish Build Artifact	7s
Azure Web App	38s
Post-job: Checkout N...	< 1s
Finalize Job	< 1s
Report build status	< 1s

Al completarse la pull request que teníamos pendiente, **se ejecutará el pipeline y veremos el resultado de las tasks creadas en el yaml. Como podemos ver, se han ejecutado correctamente**



The screenshot displays the Azure Pipelines interface. On the left, the 'Release' section shows a 'Manually triggered' build by Alba Cotoira Franco on 2/4/2025, 18:47. Below it, artifacts are listed for the build. The 'Stages' section shows a 'DEV' stage that has 'Succeeded' on 2/4/2025, 18:48. On the right, a detailed view of the 'DEV' stage is shown, including a 'Summary' tab and a 'Commits' tab. The 'Commits' tab lists several commits, including 'Merged PR 18: Set up CI with Azure Pipelines' and 'First version of API'. A 'Work Items' tab is also visible, showing a task 'Task 19: Crear Pipeline para desplegar'.

En la sección de Releases, podríamos hacer un seguimiento a partir de la versión que se ha desplegado, y relacionarla así con los Commits o incluso con los Work Items asociados

Azure Pipelines: Programación de despliegues

<NetCoreConf/>
Barcelona '25



The screenshot displays the Azure Pipelines configuration interface. On the left, a sidebar menu includes options like 'Manage security', 'Rename/move', 'Status badge', 'Trigger issues', 'Settings', 'Delete', and 'Scheduled runs' (which is highlighted with a red circle). The main area shows the YAML configuration for a pipeline with two scheduled builds: a 'Daily midnight build' and a 'Weekly Sunday build'. On the right, a 'Scheduled runs' panel lists the next four scheduled runs for the pipeline, all at 7:00:00 PM on specific dates in January 2020.

```
YAML

schedules:
- cron: '0 0 * * *'
  displayName: Daily midnight build
  branches:
    include:
      - main
      - releases/*
    exclude:
      - releases/ancient/*
- cron: '0 12 * * 0'
  displayName: Weekly Sunday build
  branches:
    include:
      - releases/*
  always: true
```

Scheduled runs

The next few runs of this pipeline scheduled for this week are shown below. See schedules to diagnose problems with scheduled runs.

Run Date and Time	Branch
1/8/2020, 7:00:00 PM	Prefs/heads/main
1/9/2020, 7:00:00 PM	Prefs/heads/main
1/10/2020, 7:00:00 PM	Prefs/heads/main
1/11/2020, 7:00:00 PM	Prefs/heads/main

¿Qué más cosas podemos hacer con Azure Pipelines, más concretamente con los builds?

Pues se **pueden crear programaciones** para los despliegues o las validaciones. Podemos añadir una programación o Schedule a nuestro yaml, y luego, en la sección de Scheduled Runs del pipeline, ver las ejecuciones.

- ¿Qué son?
 - Un agente es una infraestructura informática con software de agente instalado que ejecuta un trabajo de uno en uno
- Tipos
 - Hospedados y administrados por Microsoft
 - Auto-hospedados: configurados y hospedados en las máquinas virtuales
 - De conjunto de escalado o de máquinas virtuales de Azure: Son del tipo anterior pero pueden escalar automáticamente
 - De grupos DevOps administrados: residen en una suscripción de Microsoft Azure y no en nuestra suscripción de Azure

¿Qué son los agentes?

Son una infraestructura informática con software instalado que ejecuta un trabajo de uno en uno

Para compilar el código o implementar el software mediante Azure Pipelines, necesitamos al menos un agente. A medida que crezca el código base y el equipo, necesitaremos más agentes.



Project Settings
NetCoreBCN2025

General

Overview

Teams

Permissions

Notifications

Service hooks

Dashboards

Boards

Project configuration

Team configuration

GitHub connections

Pipelines

Agent pools

Parallel jobs

Project details

Name

Description

Este Proyecto gestionará un API en .Net8 para creación y consulta de pedido de un restaurante. Se utilizará como ejemplo en la charla del Máster de la NetCoreConf2025

Process
Agile

Visibility

This determines who can view this project. Learn more about project visibility.

Save

Get the agent

Windows

macOS

Linux

x64

x86

System prerequisites
Configure your account by following the steps outlined here.

Download the agent

Download

Create the agent

```
PS C:\> mkdir agent ; cd agent
PS C:\agent> Add-Type -AssemblyName System.IO.Compression.FileSystem ;
[System.IO.Compression.ZipFile]::ExtractToDirectory("$HOME\Downloads\vsts-agent-win-x64-4.252.0.zip", "$PWD")
```

Configure the agent Detailed instructions

```
PS C:\agent> .\config.cmd
```

Optionally run the agent interactively
If you didn't run as a service above:

```
PS C:\agent> .\run.cmd
```

That's it!
More information



Default

Jobs Agents Details Security Approvals

Name

TKTLTP155b

● Offline

Name

TKTLTP155b

● Online

System capabilities

Name	Value
Agent.Name	TKTLTP155b
Agent.Version	4.252.0
Agent.ComputerName	TKTLTP155
Agent.HomeDirectory	C:\vs-agent-win-x64-4.252.0
Agent.OS	Windows_NT
Agent.OSArchitecture	x64
Agent.OSVersion	10.0.22H2
ALLUSERSPROFILE	C:\ProgramData
APPDATA	C:\Users\AlbertoCervera\AppData\Roaming
AzurePS	5.7.0
Cmd	C:\WINDOWS\system32\cmd.exe
CommonProgramFiles	C:\Program Files\Common Files
CommonProgramFiles(x86)	C:\Program Files (x86)\Common Files
CommonProgramFiles64	C:\Program Files\Common Files
COMPUTERNAME	TKTLTP155
ComSpec	C:\WINDOWS\system32\cmd.exe
docker	C:\Program Files\Docker\Docker\resources\bin\docker.exe
DotNetFramework	C:\Windows\Microsoft.NET\Framework\v4.0.30319
DotNetFramework, 2.0	C:\Windows\Microsoft.NET\Framework\v2.0.50727
DotNetFramework, 2.0, x64	C:\Windows\Microsoft.NET\Framework64\v2.0.50727
DotNetFramework, 3.0	C:\Windows\Microsoft.NET\Framework\v3.0
DotNetFramework, 3.0, x64	C:\Windows\Microsoft.NET\Framework64\v3.0

Update all agents

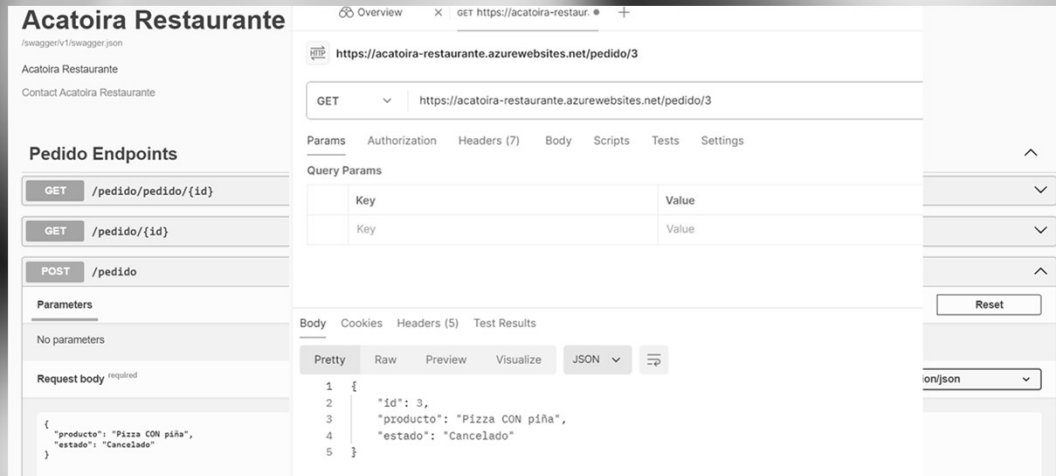
New agent

Enabled

● On

● On

Y si accedemos a los detalles del agente, podremos ver sus características



Al haber terminado de desplegarse, ya tenemos nuestro API disponible, y podríamos hacer peticiones desde swagger, por ejemplo, si lo tenemos configurado o desde Postman

Chiste con Leo??????

Aquí hemos usado swagger para cometer un sacrilegio y pedir una Pizza con piña, pero como Leo es un hater de la piña en la pizza, hemos cancelado el pedido. Y hemos comprobado el estado del pedido mediante una petición Postman.

Con todo esto, ya tenemos nuestro API desplegado, nuestro código en un repositoria Azure, y un backlog donde hacer un seguimiento del trabajo. ¿Qué nos faltaría?



Test Plans

- ¿Qué es?

- Es una herramienta para gestionar diferentes tipos de pruebas que permite:

- **Realizar pruebas manuales y exploratorias:** permite organizar las pruebas en planes de prueba y conjuntos de pruebas, realizar pruebas de aceptación de usuario y recopilar comentarios de las partes interesadas
- **Automatización de pruebas:** integrar Azure Test Plans con Azure Pipelines para admitir pruebas dentro de CI/CD. Asocie planes de prueba y casos de prueba con canalizaciones de compilación o versión.
- **Garantizar la rastreabilidad:** permite vincular casos de prueba y conjuntos de pruebas a casos de usuario, Vincule automáticamente las pruebas y defectos a los requisitos y compilaciones que se están probando.
- **Seguimiento de informes y análisis:** supervisar los resultados de las pruebas y el progreso con gráficos de seguimiento, widgets específicos, y el servicio Analytics.

Let me check



Es una herramienta para gestionar diferentes tipos de pruebas que permite:

- 1) Pruebas manuales:
- 2) Automatización de pruebas: permite integrar Azure Test Plans con Azure Pipelines o revisar los resultados de las pruebas a través de informes de progreso integrados
- 3) En cuanto a la rastreabilidad, permite vincular casos de prueba a historias de usuario
- 4) Permite supervisar los resultados con gráficos de seguimiento

Test Plans: es la herramienta que te permite conocer los resultados de las pruebas que has realizado en tu proyecto. Pueden ser los resultados de una batería de pruebas realizadas por un tester, o puede ser el resultado de las pruebas que ha ejecutado tu pipeline. En nuestro ejemplo nos vamos a centrar en este segundo caso: el de las pruebas automatizadas

←

Sprint 1

★

⋮

Oct 27 - Nov 3

Past

6% run, 0% passed. View report

Test Suites

+

-

▼ Sprint 1

33 : Change initial view (7)

34 : Welcome back pa... (...)

43 : Cancel order form (1)

33 : Change initial view (ID: 68)

ⓘ Help

Define

Execute

Chart

↓

↑

⌵

⌶

↗

⌵

Test Cases (7 items)

New Test Case

▼

☐ Title	Order	Test Case Id
☐ Test welcome page	1	76
☐ Change initial view	2	79
☐ Welcome back	3	80
☐ Resume	4	81
☐ Interim save on long form	5	82
☐ Change colors on initial view	6	83
☐ Revert to previous version	7	84

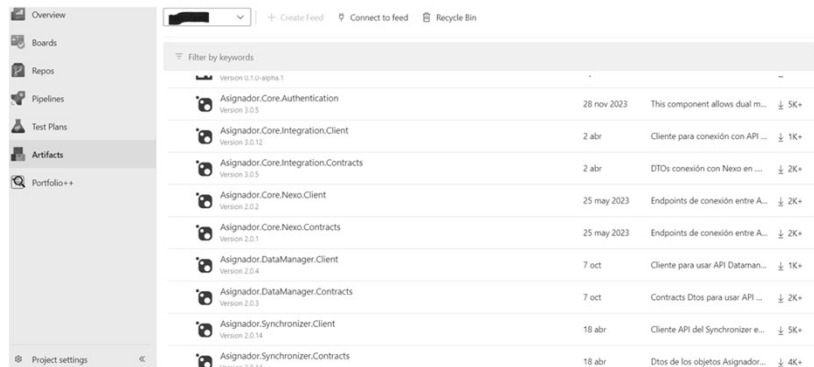
Este sería un ejemplo de los test cases definidos, asociados a un sprint en concreto



Artifacts

- ¿Qué es?

- Azure Artifacts permite a los desarrolladores administrar dependencias en un solo lugar, por ejemplo: Publicar paquetes para fuentes y compartirlos en equipos, entre organizaciones o públicamente



Filter by keywords			
Version 0.1.0-alpha.1			
Asignador.Core.Authentication Version 3.0.5	28 nov 2023	This component allows dual m...	↓ 5K+
Asignador.Core.Integration.Client Version 3.0.12	2 abr	Cliente para conexión con API ...	↓ 1K+
Asignador.Core.Integration.Contracts Version 3.0.5	2 abr	DTOs conexión con Nexo en ...	↓ 2K+
Asignador.Core.Nexo.Client Version 2.0.2	25 may 2023	Endpoints de conexión entre A...	↓ 2K+
Asignador.Core.Nexo.Contracts Version 2.0.1	25 may 2023	Endpoints de conexión entre A...	↓ 2K+
Asignador.DataManager.Client Version 2.0.4	7 oct	Cliente para usar API Dataman...	↓ 1K+
Asignador.DataManager.Contracts Version 2.0.3	7 oct	Contracts Dtos para usar API ...	↓ 2K+
Asignador.Synchronizer.Client Version 2.0.14	18 abr	Cliente API del Synchronizer e...	↓ 5K+
Asignador.Synchronizer.Contracts Version 2.0.14	18 abr	Dtos de los objetos Asignador...	↓ 4K+

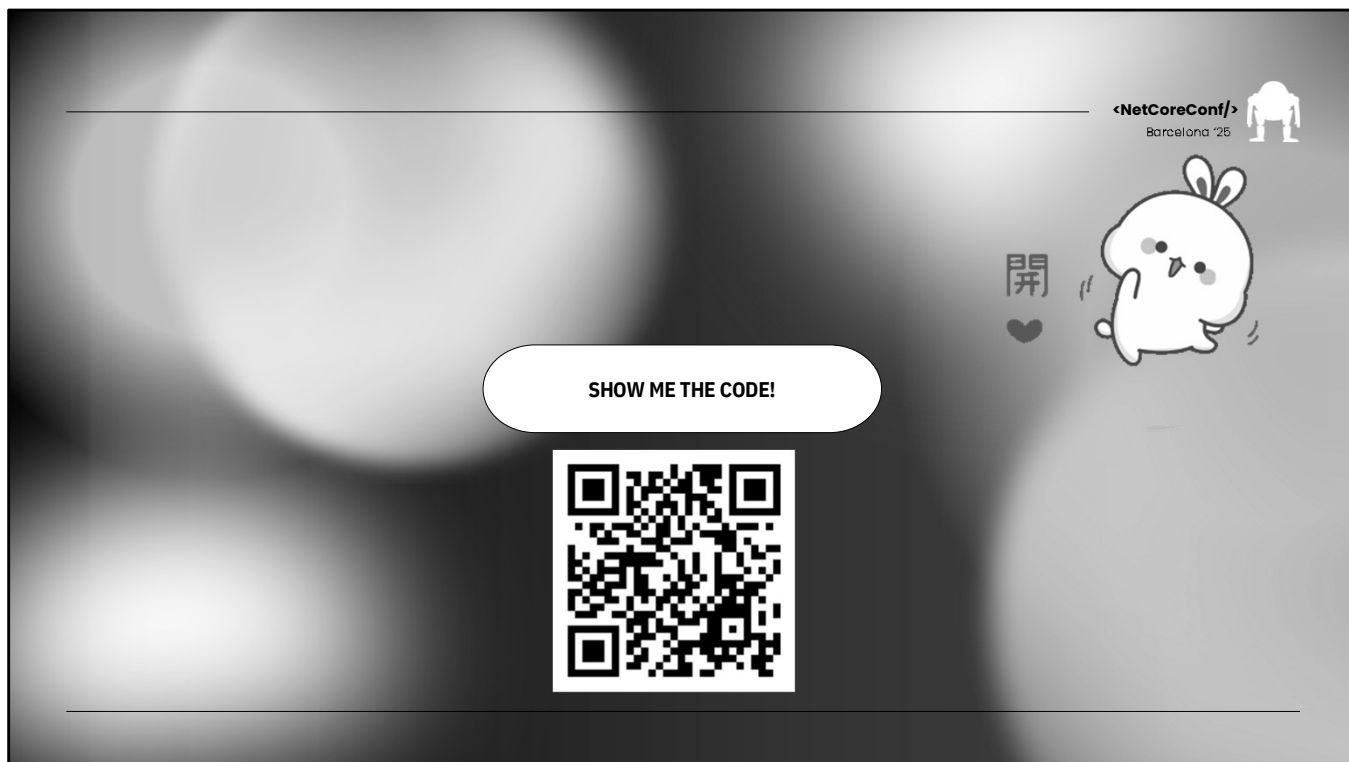


Azure Artifacts permite crear paquetes dentro de la organización fácilmente instalables desde nuget añadiendo la conexión al feed correspondiente

Azure Artifacts es una herramienta más aislada, porque no todos los proyectos generan un artefacto.

De hecho, nuestro proyecto de API no lo genera. Esta herramienta se usa en caso de querer generar un paquete para, por ejemplo, nuget o npm

¿Qué más cosas hay en Azure DevOps? Pues hay también una Wiki donde rellenar toda clase de documentación, y que puede ser de código o versionada



Y con esto, hemos terminado esta presentación, donde hemos podido configurar los aspectos básicos de un proyecto en Azure Devops, y espero que os animéis a probar

Sponsors

<NetCoreConf/>
Barcelona '25



axazure
by avvale

devoteam

ilitia

Inkoova

NTT DATA

plain
concepts

prodware

intelequia

SCRM⁺
INTERNATIONAL HUB

TOKIOTA

V-Valley
enhancing your business

Venue Sponsors

In collaboration with

Centre d'Estudis
Politécnics

sessionize

Microsoft



¡Gracias!



alba.catoira@tokiota.com



leonardo.micheloni@tokiota.com
